2573. Find the String with LCP

Hard | Topics | Companies | Hint

We define the `lcp` matrix of any **0-indexed** string `word` of `n` lowercase English letters as an `n x n` grid such that:

- `lcp[i][j]` is equal to the length of the **longest common prefix** between the substrings `word[i,n-1]` and `word[j,n-1]`.

Given an `n x n` matrix `lcp`, return the alphabetically smallest string `word` that corresponds to `lcp`. If there is no such string, return an empty string.

A string `a` is lexicographically smaller than a string `b` (of the same length) if in the first position where `a` and `b` differ, string `a` has a letter that appears earlier in the alphabet than the corresponding letter in `b`. For example, `"aabd"` is lexicographically smaller than `"aaca"` because the first position they differ is at the third letter, and `'b'` comes before `'c'`.

Example 1:

```
Input: lcp = [[4,0,2,0],[0,3,0,1],[2,0,2,0],[0,1,0,1]]
Output: "abab"
Explanation: lcp corresponds to any 4 letter string with two alternating
letters. The lexicographically smallest of them is "abab".
```

Example 2:

```
Input: lcp = [[4,3,2,1],[3,3,2,1],[2,2,2,1],[1,1,1,1]]
Output: "aaaa"
Explanation: lcp corresponds to any 4 letter string with a single
distinct letter. The lexicographically smallest of them is "aaaa".
```

Example 3:

```
Input: lcp = [[4,3,2,1],[3,3,2,1],[2,2,2,1],[1,1,1,3]]
Output: ""
Explanation: lcp[3][3] cannot be equal to 3 since word[3,...,3] consists
of only a single letter; Thus, no answer exists.
```

Constraints:

- `1 <= n == lcp.length == lcp[i].length <= 1000`
- `0 <= lcp[i][j] <= n`

Python:

```
import numpy as np

class Solution:
    def findTheString(self, lcp: List[List[int]]) -> str:
        n = len(lcp)
        L = np.array(lcp, dtype=float)

        # Diagonal must equal n - i (suffix length at position i)
        for i in range(n):
            if lcp[i][i] != n - i:
                return ""

        E = (L > 0).astype(float)

        res = [0] * n
        current_char = 1

        for i in range(n):
            if res[i] > 0: continue
            if current_char > 26: return ""

            v = E[:, i]
            clique_indices = np.where(v > 0.5)[0]

            for idx in clique_indices:
                if res[idx] == 0:
                    res[idx] = current_char
                elif res[idx] != current_char:
                    return ""

            current_char += 1

        word = "".join(chr(ord('a') + c - 1) for c in res)

        for i in range(n - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                expected = 0
                if word[i] == word[j]:
                    expected = 1 + (lcp[i+1][j+1] if i+1 < n and j+1 < n else 0)
                if lcp[i][j] != expected:
                    return ""
```

```
    return word
```

JavaScript:

```
/*************************😎*************************/
// Utility to initialize 2D array
const initialize2DArray = (rows, cols) =>
  Array.from({ length: rows }, () => Array(cols).fill(0));

// Convert number to character
const char = (code) => String.fromCharCode(code);

// Main function
const findTheString = (matrix) => {
  const n = matrix.length;

  // Step 1: Initialize structures
  const group = Array(n).fill(-1);
  let nextGroup = 1;
  const lcp = initialize2DArray(n, n);

  group[0] = 0;

  // Step 2: Assign group indices
  for (let i = 1; i < n; i++) {
    let found = false;

    for (let j = 0; j < i; j++) {
      if (matrix[i][j] > 0) {
        group[i] = group[j];
        found = true;
        break;
      }
    }

    if (!found) {
      group[i] = nextGroup;
      if (group[i] >= 26) return "";
      nextGroup++;
    }
  }

  // Step 3: Build LCP matrix
  lcp[n - 1][n - 1] = 1;
```

```
  for (let i = n - 2; i >= 0; i--) {
    for (let j = i; j < n; j++) {
      if (group[i] !== group[j]) {
        lcp[i][j] = 0;
      } else {
        lcp[i][j] = j + 1 < n ? lcp[i + 1][j + 1] + 1 : 1;
      }
    }
  }

  // Step 4: Validate matrix
  for (let i = 0; i < n; i++) {
    for (let j = i; j < n; j++) {
      if (
        lcp[i][j] !== matrix[i][j] ||
        lcp[i][j] !== matrix[j][i]
      ) {
        return "";
      }
    }
  }

  // Step 5: Convert group indices to characters
  return group
    .map((g) => String.fromCharCode(g + 97)) // 97 = 'a'
    .join("");
};
```

Java:

```
class Solution {
        public String findTheString(int[][] lcp) {
        int n = lcp.length, c = 0, A[] = new int[n];
        for (int i = 0; i < n; ++i) {
            if (A[i] > 0) continue;
            if (++c > 26) return "";
            for (int j = i; j < n; ++j)
                if (lcp[i][j] > 0)
                    A[j] = c;
        }
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                int v = i + 1 < n && j + 1 < n ? lcp[i + 1][j + 1] : 0;
                v = A[i] == A[j] ? v + 1 : 0;
                if (lcp[i][j] != v) return "";
```

```
            }
        }
        StringBuilder res = new StringBuilder();
        for (int a : A)
            res.append((char)('a' + a - 1));
        return res.toString();
    }
}
```